从分块到线段树（二）

值域分块

Points

Codeforces 19D

给你一个平面直角坐标系。处理 n 个操作，操作有三种

- `add x y`：给点 (x, y) 打上标记。保证此时点 (x, y) 上没有标记。
- `remove x y`：擦除点 (x, y) 上的标记。保证此时点 (x, y) 上有标记。
- `find x y`：检查是否有被标记的点 (x', y') 满足 $x' > x$ 且 $y' > y$。若有，输出一个这样的点的坐标。若有多个，输出字典序最小的坐标。若没有，输出 -1。

$1 \le n \le 2 \cdot 10^5$，点的坐标是不超过 10^9 的非负整数。

离线处理操作

- 读取 n 个操作。
- 对读入的 n 个点的横坐标进行压缩。设压缩过后点的横坐标的范围是 $[0, N)$。

分块

- 取 $B = \lfloor \sqrt{N} \rfloor$，对横坐标的取值范围进行分块。这种分块我们称之为**值域分块**。
- 对每个块 i，我们维护 `b[i]`：横坐标在块 i 里的那些点的纵坐标的最大值。
- 对每个横坐标 j，用一个 set<int> `a[j]` 存储横坐标等于 j 的那些点的纵坐标。

区间查询

对于询问 `find x y`，我们要找出 $[x+1, N)$ 范围内第一个满足下述条件的横坐标 j

- `a[j]` 里的最大元素大于 y

为了快速找出这个横坐标，

- 对于边块，逐个枚举其中相关的横坐标 j，检查 `a[j]` 的最大元素是否大于 y。
- 对于整块 i，如果 `b[i]` 大于 y，就逐个检查其中的横坐标。

当我们得到所求的横坐标 j 后，再查找 `a[j]` 里第一个大于 y 的值即可。

时间：$O(\sqrt{N})$。

单点修改

`add x y`：

```
a[x].insert(y);
b[x / B] = max(b[x / B], *a[x].rbegin());
```

时间：$O(\log n)$。

`remove x y`：

```
a[x].remove(y);
// 重新计算 b[x / B]
int ib = x / B;
int l = ib * B, r = min(N, l + B);
b[ib] = -1;
for (int i = l; i < r; i++)
    if (!a[i].empty())
        b[ib] = max(b[ib], *a[i].rbegin());
```

时间：$O(B) = O(\sqrt{N}) = O(\sqrt{n})$。

完整代码

```
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    vector<string> type(n);
    vector<int> x(n), y(n);
    for (int i = 0; i < n; i++)
        cin >> type[i] >> x[i] >> y[i];

    vector<int> real_x = compress(x);

    int N = (int) real_x.size();
    int B = (int) sqrt(N);
    int NB = (N + B - 1) / B;

    vector<set<int>> a(N);
    vector<int> b(NB, -1);
```

```
auto query = [&](int l, int r, int y) -> int {
    int lb = (l + B - 1) / B;
    int rb = r / B;
    if (lb > rb) {
        for (int i = l; i < r; i++)
            if (!a[i].empty() && *a[i].rbegin() > y)
                return i;
        return r;
    }

    for (int i = l; i < lb * B; i++)
        if (!a[i].empty() && *a[i].rbegin() > y)
            return i;

    for (int ib = lb; ib < rb; ib++)
        if (b[ib] > y)
            for (int i = ib * B; i < (ib + 1) * B; i++)
                if (!a[i].empty() && *a[i].rbegin() > y)
                    return i;

    for (int i = rb * B; i < r; i++)
        if (!a[i].empty() && *a[i].rbegin() > y)
            return i;

    return r;
};
```

```
    for (int i = 0; i < n; i++) {
        if (type[i] == "add") {
            a[x[i]].insert(y[i]);
            b[x[i] / B] = max(b[x[i] / B], y[i]);
        } else if (type[i] == "remove") {
            a[x[i]].erase(y[i]);
            int ib = x[i] / B;
            int l = ib * B, r = min(l + B, N);
            // 重新计算 b[ib]
            b[ib] = -1;
            for (int j = l; j < r; j++)
                if (!a[j].empty())
                    b[ib] = max(b[ib], *a[j].rbegin());
        } else {
            int j = query(x[i] + 1, N, y);
            if (j == N)
                cout << -1 << '\n';
            else
                cout << real_x[j] << ' ' << *a[j].upper_bound(y[i]) << '\n';
        }
    }
}
```

对于每个块 i，除了维护其中的纵坐标的最大值，也可以用一个 multiset<int> `c[i]` 来维护其中的纵坐标。这样，对于 `remove x y`，我们只需要

```
a[x].erase(y);
c[x / B].extract(y); // since C++20. 只删一个 y
```

时间是 $O(\log n)$。

不过这样并不比上面的写法更块。

用线段树解决这题

```
vector<int> b(2 * N, -1);
vector<set<int>> a(N);
```

线段树的每个节点 `b[i]` 维护「横坐标在相应范围内的点」的纵坐标的最大值。

每对个横坐标 x，用一个 set<int> `a[x]` 来存储横坐标等于 x 的点的纵坐标。

单点修改

对于操作 `add x y` 或 `remove x y`，

1. 修改 `a[x]`
2. 更新线段树的叶子 `b[x + N]`
3. 从下往上，更新 `x + N` 的祖先节点

区间查询

`find x, y`：找出区间 $[x+1, N)$ 中第一个满足 `b[i + N] > y` 的 `i`。

1. 从下往上，把所查询的区间 $[x+1, N)$ 拆解为极大整块，找出从左到右第一个满足 `b[k] > y` 的极大整块 k。

   在每一层

   - 如果左边的极大整块 l 满足 `b[l] > y`，那么 l 就是 k，结束。
   - 如果右边的极大整块 $r-1$ 满足 `b[r - 1] > y`，那么 $k \leftarrow r-1$。
2. 从节点 k 开始，往下查找，定位到满足 `b[p] > y` 的叶子 p。
3. 在 `a[p - N]` 中查找第一个大于 y 的纵坐标。

我们把 1 2 两步称为**在线段树上二分查找**。

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    vector<string> type(n);
    vector<int> x(n), y(n);
    for (int i = 0; i < n; i++)
        cin >> type[i] >> x[i] >> y[i];

    vector<int> real_x = compress(x); //坐标压缩
    int N = (int) real_x.size();
    vector<set<int>> a(N);
    vector<int> b(2 * N, -1); //线段树

    for (int i = 0; i < n; i++) {
        if (type[i] == "add") {
            a[x[i]].insert(y[i]);
            int p = x[i] + N;
            b[p] = *a[x[i]].rbegin();
            for (p /= 2; p > 0; p /= 2) {
                b[p] = max(b[p * 2], b[p * 2 + 1]);
            }
        } else if (type[i] == "remove") {
            a[x[i]].erase(y[i]);
            int p = x[i] + N;
            b[p] = a[x[i]].empty() ? -1 : *a[x[i]].rbegin();
            for (p /= 2; p > 0; p /= 2) {
                b[p] = max(b[p * 2], b[p * 2 + 1]);
            }
        } else {
            int l = x[i] + 1 + N, r = N + N;
            int p = -1;
            while (l < r) {
                if (l & 1) {
                    if (b[l] > y[i]) { p = l; break; }
                    l++;
                }
                if (r & 1) {
                    if (b[r - 1] > y[i]) p = r - 1;
                    r--;
                }
                l /= 2; r /= 2;
            }
            if (p == -1)
                cout << -1 << '\n';
            else {
                while (p < N) {
                    if (b[p * 2] > y[i])
                        p = 2 * p;
                    else
                        p = 2 * p + 1;
                }
                cout << real_x[p - N] << ' ' <<
                    *a[p - N].upper_bound(y[i]) << '\n';
            }
        }
    }
}
```

LCM

代码源挑战赛 R53F

定义 $f(x)$ 为使得从 1 到 y 的所有正整数都能整除 x 的最大正整数 y。

例如 $f(15) = 3$，因为 $1, 2, 3$ 都能整除 15，但 4 不整除 15。

给定一个长为 n 的正整数序列 $a = (a_1, \ldots, a_n)$。回答 q 个询问

- `l r`：计算 $f(\mathrm{lcm}(a_l, \ldots, a_r))$。

$\mathrm{lcm}(a_l, \ldots, a_r)$ 表示 $a_l, \ldots, a_r$ 的最小公倍数。比如 $\mathrm{lcm}(2, 4, 6, 8) = 24$。

$1 \le n, q \le 2 \cdot 10^5$，$1 \le a_i \le 10^6$

分析

对于正整数 x，我们来考虑 $f(x)+1$。也就是满足下述条件的最小正整数 k

- $1, 2, \ldots, k-1$ 都能整除 x 而 k 不能整除 x。

我们把这个数记作 $g(x)$。

k 一定是某个素数的幂，形如 p^e。我们把这样的数称为**素数幂**。

因为$a_i \le 10^6$，所以 $g(\mathrm{lcm}(a_l, \ldots, a_r))$ 不超过「大于 10^6 第一个素数幂」。

经过计算我们知道

- 「大于 10^6 第一个素数幂」是 $10^6 + 3$。
- $2, 3, 4, \ldots, 10^6 + 3$ 中的素数幂一共有 78735 个。

> 其实 $10^6 + 3$ 是素数。

考虑不超过 10^6 的素数幂构成的序列 q，它的长度是 78734。
$q_0 = 2, q_1 = 3, \ldots, q_{78733} = 999983$。

离线回答询问

按 r 从小到大的顺序回答询问。

在回答询问 (l, r) 之前，我们维护一个序列，pos[0], pos[1], ..., pos[78733]。
pos[i]：$a_1, \ldots, a_r$ 中最后一个能被第 i 个素数幂 q_i 整除的数的下标。

或者说 pos[i] 是 $a_1, \ldots, a_r$ 中「最后一个 q_i 的倍数」的下标。

我们也可以说，pos[i] 就是第 i 个素数幂 q_i 在 $a_1, \ldots, a_r$ 中最后一次出现的位置。

如果 $a_1, \ldots, a_r$ 中没有 q_i 的倍数，就让 pos[i] = 0。

设 j 是第一个满足 pos[i] < l 的下标 i，那么

- 第一个不能整除 $\mathrm{lcm}(a_l, \ldots, a_r)$ 的数就是 q_j。

线段树

对序列 pos 建线段树，线段树的节点维护块内的 pos[i] 的最小值。

序列的下标，0 到 78733，其实是我们所考虑的问题的答案的取值范围，或者说值域，所以我们把这样的线段树称为**值域线段树**。

我们可以在值域线段树进行二分查找来找出

- 满足 pos[i] < l 的第一个下标 i。

```
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int maxn = 1 << 20;

    vector<vector<int>> f(maxn + 1); // f[i]：能整除 i 的素数幂
    vector<int> np(maxn + 1); // number of prime factors
    vector<int> val;
    for (int i = 2; ; i++) {
        if (f[i].empty()) {
            for (int j = i; j <= maxn; j += i) {
                np[j]++;
                int t = j;
                int d = 1;
                do {
                    d *= i;
                    t /= i;
                    f[j].push_back(d);
                } while (t % i == 0);
            }
        }
        if (np[i] == 1) {
            val.push_back(i);
            if (i > (int) 1e6)
                break;
        }
    }

    vector<int> order(maxn + 1);
    int N = (int) val.size() - 1; // N = 78734
    for (int i = 0; i < N; i++)
        order[val[i]] = i;
```

```
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    vector<int> L(q), R(q);
    for (int i = 0; i < q; i++) {
        cin >> L[i] >> R[i];
        L[i]--;
        R[i]--;
    }
    vector<int> I(q);
    iota(I.begin(), I.end(), 0);
    sort(I.begin(), I.end(), [&](int i, int j) {return R[i] < R[j];});
    vector<int> ans(q);
```

```
    auto it = I.begin();
    for (int i = 0; i < n; i++) {
        for (int d : f[a[i]]) {
            int p = order[d] + N;
            pos[p] = i;
            for (p >>= 1; p; p >>= 1) {
                pos[p] = min(pos[2 * p], pos[2 * p + 1]);
            }
        }
```

```
        for ( ;it != I.end() && R[*it] == i; ++it) {
            int l = N, r = 2 * N;
            int p = r;
            while (l < r) {
                if (l & 1) {
                    if (pos[l] < L[*it]) {
                        p = l;
                        break;
                    }
                    l++;
                }
                if (r & 1) {
                    if (pos[r - 1] < L[*it])
                        p = r - 1;
                    r--;
                }
                l >>= 1;
                r >>= 1;
            }
            while (p < N) {
                if (pos[p * 2] < L[*it])
                    p = p * 2;
                else
                    p = p * 2 + 1;
            }
            ans[*it] = val[p - N] - 1;
        }
    }
    for (int x : ans)
        cout << x << '\n';
}
```

另一个写法

```
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int maxn = 1 << 20;

    vector<vector<int>> f(maxn + 1);
    vector<int> np(maxn + 1); // number of prime factors
    vector<int> val;
    for (int i = 2; ; i++) {
        if (np[i] <= 1) { // i 是素数幂
            bool is_prime = np[i] == 0;
            for (int j = i; j <= maxn; j += i) {
                np[j] += is_prime;
                f[j].push_back((int) val.size());
            }
            val.push_back(i);
            if (i > (int) 1e6)
                break;
        }
    }

    int N = (int) val.size() - 1; // N = 78734
    vector<int> pos(2 * N, -1); // 线段树
```

```
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    vector<int> L(q), R(q);
    for (int i = 0; i < q; i++) {
        cin >> L[i] >> R[i];
        L[i]--;
        R[i]--;
    }
    vector<int> I(q);
    iota(I.begin(), I.end(), 0);
    sort(I.begin(), I.end(), [&](int i, int j) {return R[i] < R[j];});

    vector<int> ans(q);
```

```
    auto it = I.begin();
    for (int i = 0; i < n; i++) {
        for (int d : f[a[i]]) {
            int p = d + N;
            pos[p] = i;
            for (p >>= 1; p; p >>= 1) {
                pos[p] = min(pos[2 * p], pos[2 * p + 1]);
            }
        }
```

```
        for ( ;it != I.end() && R[*it] == i; ++it) {
            int l = N, r = 2 * N;
            int p = r;
            while (l < r) {
                if (l & 1) {
                    if (pos[l] < L[*it]) {
                        p = l;
                        break;
                    }
                    l++;
                }
                if (r & 1) {
                    if (pos[r - 1] < L[*it])
                        p = r - 1;
                    r--;
                }
                l >>= 1;
                r >>= 1;
            }
            while (p < N) {
                if (pos[p * 2] < L[*it])
                    p = p * 2;
                else
                    p = p * 2 + 1;
            }
            ans[*it] = val[p - N] - 1;
        }
    }
    for (int x : ans)
        cout << x << '\n';
}
```